CS316 Chapter 2.1: Pandas Code Sheet

Scenario: Riyadh Reads bookstore sales data (CSV, Excel, SQL, JSON)

1. Theory in brief

Topic	Key points
Pandas	Python library for data manipulation and analysis, built on NumPy. Simplifies cleaning, exploring and transforming data.
vs Excel	Better for automated, repeatable workflows and larger datasets.
vs SQL	SQL queries databases; pandas analyzes data in memory. They combine via pd.read_sql().
vs NumPy	Built on NumPy; adds easier handling of tabular data.
Series	1D labeled array (one column). Values aligned to an index. Can hold mixed types. Vectorized operations.
DataFrame	2D labeled table (rows + columns). Columns can have different types. Size-mutable. Several Series sharing one index.
Key functions	Missing data: .isnull() .dropna() .fillna() | Grouping: .groupby() .agg() | Combining: merge() join()

2. Setup

```
import pandas as pd
import os
os.makedirs("bookstore_data", exist_ok=True)   # create folder; no error if it exists
```

3. Series: create

```
prices = pd.Series([45.0, 60.0, 35.5])                    # default index 0, 1, 2
book_prices = pd.Series([45.0, 60.0, 35.5],
                        index=["Atomic Habits", "Dune", "The Alchemist"],
                        name="prices")                    # custom labels + a name
stock = pd.Series({"Atomic Habits": 12, "Dune": 5})       # dict: keys become the index
```

4. Series: access and slice

```
book_prices["Dune"]              # by label        -> 60.0
book_prices.iloc[0]              # by position     -> 45.0
book_prices.iloc[0:3]            # positions 0, 1, 2   (end EXCLUDED)
book_prices["Dune":"Sapiens"]    # by label            (end INCLUDED)
book_prices.iloc[::2]            # every 2nd item (positions 0, 2, 4)
book_prices.iloc[1::2]           # every 2nd item starting at position 1
book_prices.iloc[::-1]           # reversed
```

5. DataFrame: create

```
books_df = pd.DataFrame({"price": book_prices,
                         "stock": stock})                 # from Series: keys = column names
sales_df = pd.DataFrame({"title": ["Dune", "1984"],
                         "units_sold": [1, 4],
                         "unit_price": [60.0, 25.0]})     # from a dict of lists
sales_df["revenue"] = sales_df["units_sold"] * sales_df["unit_price"]   # new column
```

6. DataFrame: access, select, filter

```
df["title"]                       # one column       -> Series
df[["title", "revenue"]]          # list of columns  -> DataFrame
df[["title"]]                     # still a DataFrame (double brackets)
df.head()                         # first 5 rows
df.head(2)                        # first 2 rows
df.shape                          # (rows, columns)
df.iloc[0]                        # first row by position
df.iloc[1:3]                      # rows 1, 2
df.iloc[0:3, 0:2]                 # rows 0-2, columns 0-1
df.loc[:, ["title", "revenue"]]   # all rows, chosen columns   loc[rows, cols]
df.loc[1:3]                       # rows 1, 2, 3 (label: end INCLUDED)
```

```
df.loc[df["branch"] == "Olaya"]    # filter rows
df.loc[(df["branch"] == "Olaya") & (df["revenue"] > 100)]   # AND
df.loc[(df["branch"] == "Olaya") | (df["revenue"] > 100)]   # OR
```

7. Grouping

```
summary = (df.groupby("branch", as_index=False)["revenue"].sum()
             .rename(columns={"revenue": "total_revenue"}))
# split rows by branch -> sum revenue per branch -> rename the column
# as_index=False keeps "branch" as a normal column
```

8. Reading data

```
pd.read_csv("sales.csv")
pd.read_csv("data.csv", sep="\t", header=0, index_col="ID", usecols=["Name", "Age"])
pd.read_excel("sales.xlsx", sheet_name="Sales")     # sheet_name=None -> all sheets (dict)
pd.read_excel("data.xlsx", skiprows=2, usecols="A:C")

import sqlite3
conn = sqlite3.connect("bookstore_data/bookstore.db")
pd.read_sql('SELECT * FROM sales WHERE branch="Tahlia"', conn)

pd.read_json("sales.json", orient="records")
```

Parameter	Meaning
`sep`	Delimiter (default comma; "\t" for tabs)
`header`	Which row holds the column names
`index_col`	Use a column as the index
`usecols`	Load only certain columns
`sheet_name`	Which Excel sheet (name or number)
`skiprows`	Skip rows at the start of the file

9. Exporting data

```
df.to_csv("sales.csv", index=False)
df.to_csv("out.csv", sep=";", encoding="utf-8", index=False)
df.to_excel("sales.xlsx", sheet_name="Sales", index=False)
df.to_sql("sales", conn, if_exists="replace", index=False)
conn.close()
df.to_json("sales.json", orient="records")    # one object per ROW
df.to_json("sales.json", orient="columns")    # one object per COLUMN
```

```
orient="records"  -> [{"branch":"Olaya","revenue":355.0},{"branch":"Tahlia","revenue":250.0}]
orient="columns"  -> {"branch":{"0":"Olaya","1":"Tahlia"},"revenue":{"0":355.0,"1":250.0}}
```

Format	Best for
CSV	Sharing raw data in plain text; opens in any application
Excel	Reports needing formatting or multiple sheets
SQL	Storing structured data in a database for querying
JSON	Web and API use
Large data	Export in chunks with chunksize; prefer CSV over Excel

10. File paths and URLs

```
from pathlib import Path
data_dir = Path("bookstore_data")
csv_path = data_dir / "sales.csv"      # path that works on any operating system
csv_path.exists()                      # True if the file is there
pd.read_csv(csv_path)
pd.read_csv("https://.../iris.csv")    # read straight from a URL
```

11. The 4 rules most likely to be tested

#	Rule
1	df["col"] returns a Series; df[["col"]] returns a DataFrame.
2	iloc = position, end EXCLUDED. loc = label, end INCLUDED.
3	Every condition in its own parentheses, joined with & (AND) or | (OR).
4	index=False when exporting, so row numbers are not saved as a column.

12. Worked example (sales table)

```
   title          branch  units_sold  unit_price  revenue
0  Atomic Habits  Olaya   3           45.0        135.0
1  Dune           Olaya   1           60.0         60.0
2  Atomic Habits  Tahlia  2           45.0         90.0
3  1984           Tahlia  4           25.0        100.0
4  Sapiens        Olaya   2           80.0        160.0
5  Dune           Tahlia  1           60.0         60.0
```

Code	Output
`df.shape`	(6, 5)
`df.iloc[1:3]`	2 rows: 1, 2
`df.loc[1:3]`	3 rows: 1, 2, 3
`df.loc[(df["branch"]=="Olaya") & (df["revenue"]>100)]`	Rows 0 (135) and 4 (160)
`df.groupby("branch", as_index=False)["revenue"].sum()`	Olaya 355, Tahlia 250
`df.loc[df["units_sold"] > 1].shape`	(4, 5)